

Исходная архитектура DSL содержала три отдельных класса для запросов:

- FindQuery
- ContextQuery
- DistanceQuery

Это приводило к дублированию логики в парсере, валидаторе и исполнителе, а также усложняло добавление новых типов запросов. Было принято решение объединить их в один универсальный класс Query с полем kind, а также унифицировать синтаксис, сделав общие блоки (WITHIN, WHERE, RETURN, а также новый LIMIT) одинаковыми для всех типов.

Дополнительно, для упрощения расширения, в лексер и парсер была добавлена поддержка нового ключевого слова LIMIT (вместо специфичного LIMIT_PAIRS для DISTANCE и [$\leq N$] для CONTEXT), при этом старый синтаксис сохранён для обратной совместимости.

Результат: единая грамматика, единая модель, упрощение кода, лёгкость добавления новых типов запросов в будущем.

- Модель: заменяем три класса на один Query с полями:
 - kind: Literal["FIND", "CONTEXT", "DISTANCE"]
 - source: Selector – для FIND и CONTEXT это сущность (без ограничения длины, т.к. ограничение вынесено в limit), для DISTANCE – левый селектор.
 - target: Optional[Selector] – только для DISTANCE (правый селектор).
 - patterns: Optional[List[Pattern]] – только для CONTEXT.
 - within: List[WithinConstraint]
 - where: Optional[Expression]
 - limit: Optional[Union[int, PairLimit]] – для CONTEXT целое (макс. длина окна), для DISTANCE – PairLimit, для FIND игнорируется.
 - returns: Optional[List[ReturnItem]]
- Парсер: единый метод parse_query(), который определяет тип по первому токenu и затем разбирает общие блоки в едином порядке.
- Лексер: добавлено ключевое слово LIMIT, сохранено LIMIT_PAIRS для обратной совместимости.
- Остальные компоненты (валидатор, исполнитель, акторы) адаптированы к работе с Query, используя kind для ветвления логики.

Старые классы оставлены как алиасы (`FindQuery = ContextQuery = DistanceQuery = Query`) для обеспечения совместимости с существующим кодом, использующим типы.